Lab2

Are Languages Represented in Disjoint Subspaces?
Probing the Multilingual Geometry of mBERT

Report - Lab2

Peng CHEN M2 CL 22409126 Paris Cité Université

To read the report online: Lab2

To read the code of this lab online : colab.research.google.com
Lab2 instructions: https://pages.llf-paris.fr/~gwisniewski/assets/2026/tp_mbert_subspaces.pdf

I mostly worked on Part 2. I didn’ t actually finish Part 1, I just read through it, and I also looked at the optional parts in Part 2, mainly to see how the whole lab fit together. Part 2 is about the representation geometry of different languages in mBERT, especially whether languages end up in separate subspaces. I extracted and cached sentence representations for six languages across layers, looked at norms and cosine similarities, ran logistic regression to see whether language identity was predictable from the representations, compared original and per-language centred versions, did cross-lingual retrieval, used SVD to get principal subspaces and measured their normalised overlap, and finally ran domain classification as a cross-lingual transfer test. I used AI for some of the harder code and debugging, but I tried to check the logic and intermediate results myself instead of just trusting whatever came out.

The part that took me longest was figuring out what each experiment actually tells us. I spent more time on the scientific background than I expected. A lot of results look obvious at first, but then you realise there are other explanations. The language classifier, for example, gets almost 100% accuracy on the original representations. It is very tempting to read that as “different languages use different spaces.” But after per-language centring, accuracy drops to almost chance level. So a lot of that linear separability can be explained by the mean offsets between languages. Similarly, high overlap between principal subspaces does not prove that languages share the same features, because different language-specific features could still be superposed in the same dimensions. I think that second point was the one I had to sit with for a while.

My results don’ t really support the simple “completely disjoint subspaces” story. Language identity is easy to predict before centring, but much harder after removing language-specific means. Cross-lingual retrieval is best in the middle layers, and centring usually helps it. Principal

subspace overlap is also higher in the middle layers for most language pairs, which suggests more shared geometry there. The domain probe looks similar: domain information learned on English transfers reasonably well to French, German, Russian, and Chinese, but clearly less well to Yoruba. So there seems to be a lot of shared cross-lingual structure in mBERT, but the degree of sharing depends on the language and the layer.

The thing I ended up liking about this lab was that the experiments actually build on each other. Language classification, centring, retrieval, subspace overlap, and domain transfer are not just separate scores; each one helps rule out a different possible explanation. That made representation analysis feel less like "run a probe and report accuracy" and more like asking what a result really means and what else could explain it. I also liked comparing layers, because seeing how the representations change makes the inside of a multilingual Transformer feel less like a black box. I think I learned more about how to design and interpret a sequence of experiments than about any single formula.

Part 1 — Warm-up: how unequal is tokenisation? (optional)

For this part, I just went through the questions and the overall experimental procedure, aiming to learn how to design an experiment when facing this kind of research question or hypothesis. mBERT was pre-trained on Wikipedia data from multiple languages, and all these languages share the same WordPiece vocabulary, which was also built from the pre-training corpus.

The hypothesis to be tested is that, if a language has more resources in the pre-training data, its common word forms or longer subword units may be more likely to be covered by the vocabulary. In contrast, a lower-resource language may have fewer dedicated WordPiece units, so the tokenizer may need to use more and smaller pieces to represent a word, making the tokenization more fragmented.

To test this hypothesis, in this part we first need to directly inspect how several languages are actually tokenized. For example, how many WordPieces a word is split into, what the ## prefix means, what is specific about Chinese tokenization, and why [UNK] appears for some languages.

Then, several fragmentation metrics are computed for 23 languages. More specifically,

- tokenisation premium : compares on the same set of parallel sentences, how many WordPieces a language requires relative to English;
- fertility : measures the average number of WordPieces per word; continued-word proportion measures the proportion of words that are split into two or more pieces;
- unknown rate : measures the proportion of [UNK] tokens.

After that, these tokenization results are compared with the Wikipedia resource size of each language. The main question is whether lower-resource languages tend to have a higher tokenisation premium, meaning more fragmented tokenization. A Spearman rank correlation is then computed to test whether there is a consistent ranking relationship between tokenisation

premium and Wikipedia resource size. The analysis also considers possible factors that may affect the results, such as script, [UNK], differences in tokenization across languages, and whether Wikipedia article count is itself a reliable proxy for pre-training resources.

Part 2 — Do languages share a subspace?

Task 2.1 — Extract and inspect the representations

14. Compute and cache the representations of both splits for the six languages.

Code block

```
for lang in LANGS:
    dev_path = os.path.join(CACHE_DIR, f"{lang}_dev.npy")
    devtest_path = os.path.join(CACHE_DIR, f"{lang}_devtest.npy")

    if os.path.exists(dev_path) and os.path.exists(devtest_path):
        representations_dev = np.load(dev_path)
        representations_devtest = np.load(devtest_path)
        print(lang, representations_dev.shape, representations_devtest.shape)
        continue

    representations_dev = sentence_reps(dev[lang]["text"].tolist())
    np.save(dev_path, representations_dev)

    representations_devtest = sentence_reps(devtest[lang]["text"].tolist())
    np.save(devtest_path, representations_devtest)

    print(lang, representations_dev.shape, representations_devtest.shape)
```

> eng_Latn (13, 997, 768) (13, 1012, 768)
>
> fra_Latn (13, 997, 768) (13, 1012, 768)
>
> deu_Latn (13, 997, 768) (13, 1012, 768)
>
> rus_Cyrl (13, 997, 768) (13, 1012, 768)
>
> cmn_Hans (13, 997, 768) (13, 1012, 768)
>
> yor_Latn (13, 997, 768) (13, 1012, 768)

15. For each layer and language, compute the average norm of the vectors and the average cosine similarity between 1,000 random pairs of distinct sentences. Plot both against the layer.

Code block

```
#exo15
```

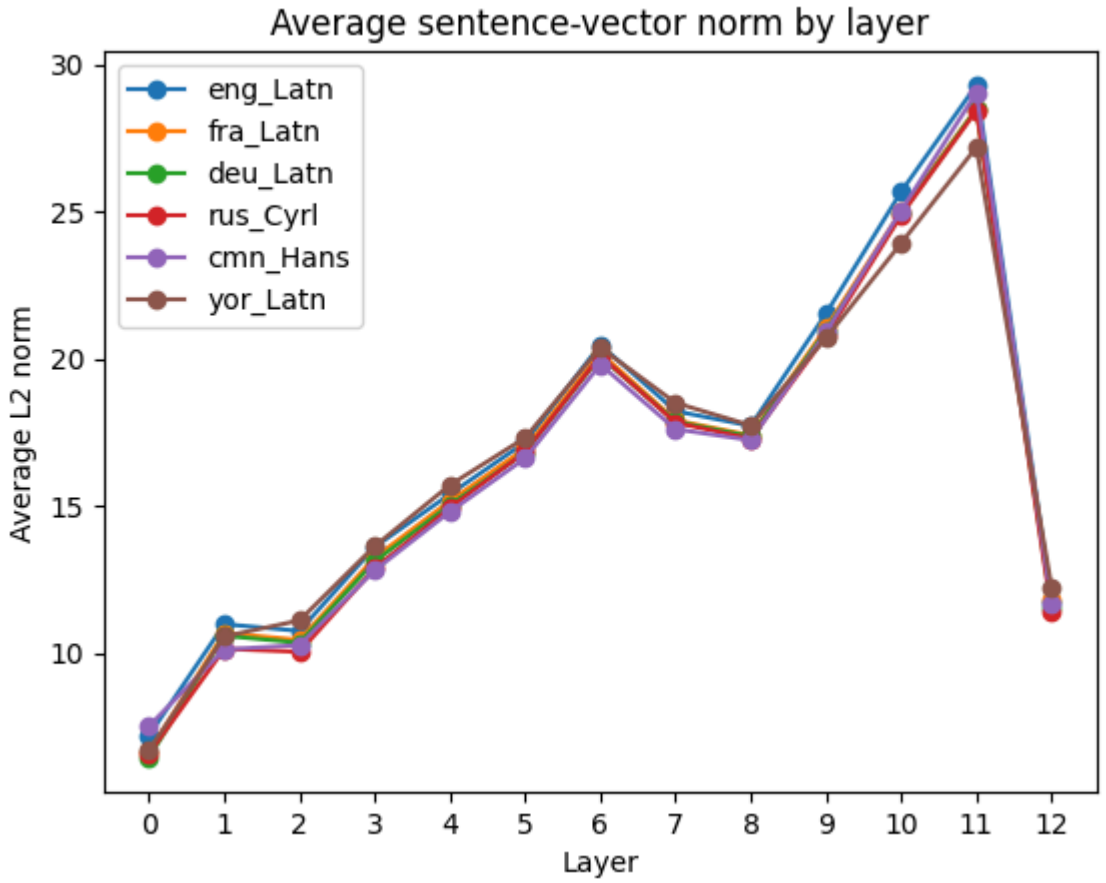
```
def compute_average_norm(vectors: np.ndarray) -> float:
  """Compute the average L2 norm of a set of sentence vectors."""
  norms = np.linalg.norm(vectors, axis=1)
  return norms.mean()

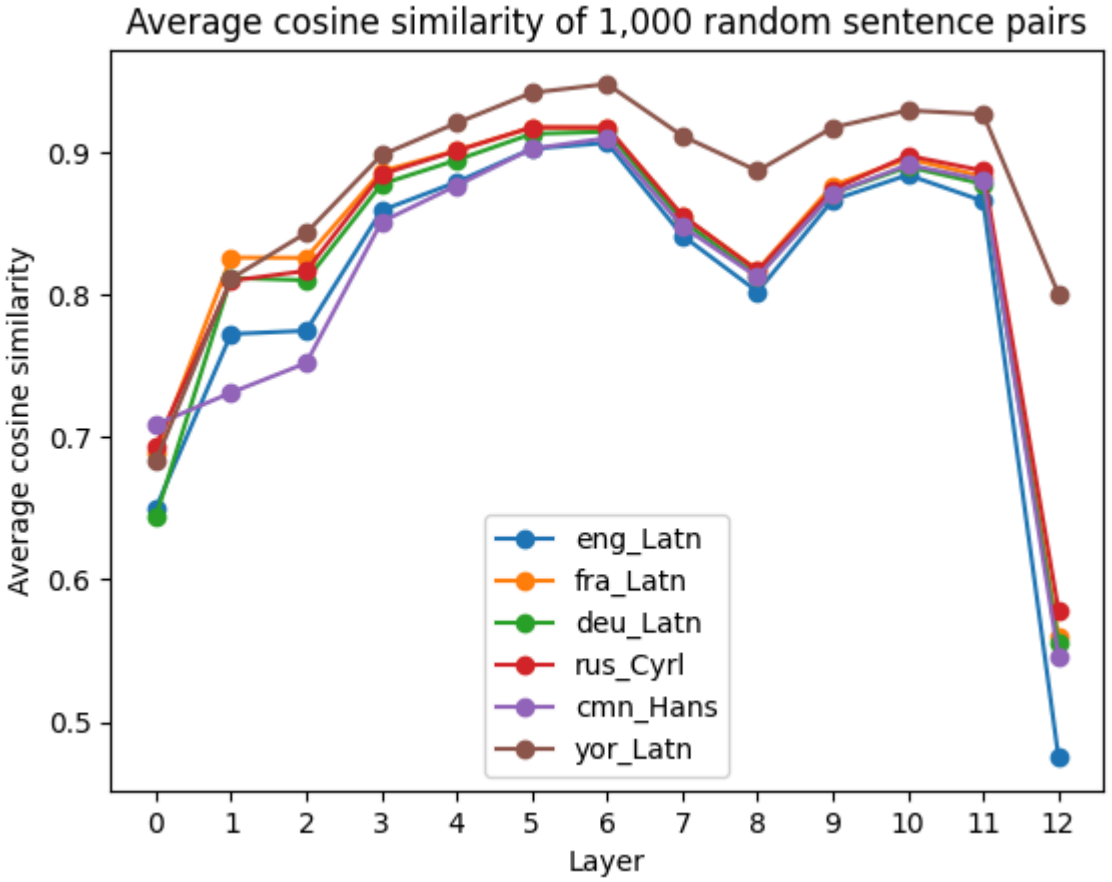
def compute_average_cosine(vectors: np.ndarray, number_of_pairs: int = 1000, seed: int = 42) -> float:
  """Compute the average cosine similarity of random pairs of distinct sentence vectors."""
  # Make sure the two sentences in each pair are different
  # ... for the details, please read the online colab or the code file.
  cosine_similarities = np.sum(first_vectors * second_vectors, axis=1) / (
        np.linalg.norm(first_vectors, axis=1) * np.linalg.norm(second_vectors, axis=1))
  return cosine_similarities.mean()

average_norms = {}
average_cosines = {}

for lang in LANGS:
  representations_dev = np.load(os.path.join(CACHE_DIR, f"{lang}_dev.npy"))
  average_norms[lang] = []
  average_cosines[lang] = []
  for layer in range(13):
      layer_vectors = representations_dev[layer]
      average_norms[lang].append(compute_average_norm(layer_vectors))
      average_cosines[lang].append(compute_average_cosine(layer_vectors))

#Plot both against the layer.
layers = range(13)
for lang in LANGS:
    plt.plot(layers, average_norms[lang], marker="o", label=lang)
plt....
for lang in LANGS:
    plt.plot(layers, average_cosines[lang], marker="o", label=lang)
plt....
```


Average sentence-vector norm by layer
Average L2 norm
Layer
eng_Latn
fra_Latn
deu_Latn
rus_Cyrl
cmn_Hans
yor_Latn
30
25
20
15
10
0
1
2
3
4
5
6
7
8
9
10
11
12


Average cosine similarity of 1,000 random sentence pairs
Average cosine similarity
Layer
eng_Latn
fra_Latn
deu_Latn
rus_Cyrl
cmn_Hans
yor_Latn
0.9
0.8
0.7
0.6
0.5
0
1
2
3
4
5
6
7
8
9
10
11
12

16. What does hidden_states[0] contain? Why can the language of a sentence be identified at this layer without any contextual processing, at least for languages with a script of their own?

hidden_states[0] contains the output of the BERT embedding layer, before any Transformer encoder layer performs contextual processing. For each WordPiece token, BERT combines its token embedding, position embedding and segment embedding, followed by layer normalisation.

The language of a sentence can already be identified at this stage, because the tokens themselves already contain language-specific information. mBERT uses a shared multilingual WordPiece vocabulary for all languages, but languages with different scripts tend to use different subsets of this vocabulary. For example, Chinese uses Han characters, which are mapped to characteristic WordPieces and token IDs. Each token ID corresponds to a different learned vector in mBERT' s embedding matrix, and these vectors are updated during pre-training according to the contexts in which the corresponding WordPieces occur.

So languages with different scripts tend to have different sets of token embeddings, providing information about the language.

17. Why are [CLS] and [SEP] excluded from the average?

[CLS] and [SEP] are just special structural tokens added by BERT. [CLS] is placed at the beginning of the input and can serve as a global representation, while [SEP] marks the end of a sentence or separates two text segments.

They still receive embeddings and participate in the Transformer computations, and could be used as a sentence representation in a different pooling strategy. BUT our sentence representation is defined as the average of the WordPiece vectors corresponding to the actual content of the sentence. And in this lab we use mean pooling over the actual WordPiece tokens, so [CLS] and [SEP] are left out, since they are not part of the actual sentence content.

18. Everything is fitted on dev and evaluated on devtest. Why would it be a mistake to pool the vectors of all languages and split them at random into a fitting set and an evaluation set?

FLORES+ is a parallel corpus, which means that sentences with the same ID in different languages are translations of the same original sentence. Therefore, if we pool the vectors of all languages together and randomly split them into a fitting set and an evaluation set, one version of a sentence, for example the English one, could be included in the fitting set, while its Chinese translation could appear in the evaluation set.

But these sentences contain essentially the same semantic content, even though they are written in different languages. So the fitting and evaluation sets would not be truly independent, and possibly make the evaluation results artificially better.

So in this lab everything is fitted on dev and evaluated on the separate devtest split, so that the evaluation is performed on unseen sentences.

19. What do your average cosine similarities suggest about the geometry of the space?

Code block

```
for lang in LANGS:
    print(lang, np.round(average_cosines[lang], 3))
```

> eng_Latn [0.65 0.772 0.775 0.859 0.879 0.902 0.907 0.842 0.802 0.866 0.884 0.866 0.476]
>
> fra_Latn [0.689 0.826 0.826 0.887 0.901 0.917 0.917 0.855 0.817 0.877 0.895 0.883 0.56]
>
> deu_Latn [0.644 0.812 0.81 0.878 0.894 0.913 0.914 0.852 0.813 0.871 0.89 0.878 0.556]
>
> rus_Cyrl [0.694 0.81 0.817 0.885 0.901 0.917 0.917 0.856 0.817 0.874 0.898 0.887 0.579]
>
> cmn_Hans [0.709 0.731 0.752 0.851 0.877 0.902 0.91 0.848 0.812 0.871 0.891 0.881 0.546]
>
> yor_Latn [0.683 0.811 0.843 0.898 0.921 0.942 0.948 0.911 0.887 0.917 0.929 0.927 0.801]

From the previous plot(exo15) and the numerical results, the average cosine similarities between random sentence pairs are quite high in most layers. For most languages, they are around 0.8–0.9 in the middle layers. Since the sentences are paired randomly, such high cosine similarities should not be interpreted as high semantic similarity between the sentences.

Instead, these results suggest that many sentence vectors tend to point in similar directions in the representation space. If the space were isotropic, the vectors would be distributed more evenly in different directions, and the average cosine similarity of random pairs would be closer to zero. Here, the consistently high values indicate a strongly anisotropic representation space.

We can also see that this effect changes across layers. At layer 0, the cosine similarities are already positive but relatively lower. They then increase through the first and middle Transformer layers, meaning that the sentence vectors become more concentrated around some common directions. In the final layer, the cosine similarity drops clearly for most languages, especially English, suggesting that the vectors become more spread out again. So the geometry of the representation space is not fixed: it changes substantially from the embedding layer to the middle and final Transformer layers.

Task 2.2 — Look for rogue dimensions (optional)

For this optional task 2.2, I just went through the questions and the experimental procedure, aiming to learn how to design an experiment when facing this kind of research question.

In the previous task, the high average cosine similarity between random sentence pairs suggested that the representation space is anisotropic. But is this anisotropy spread across many dimensions, or is it mainly caused by a few unusual rogue dimensions that have a very strong influence on cosine similarity?

The procedure first measures how much each dimension contributes to the expected cosine similarity using CC_i, and checks whether only a few dimensions dominate the result. It then compares these dominant dimensions across different languages and layers. Finally, it introduces standardisation, where each dimension is centred and scaled, to reduce the influence of these abnormal dimensions.

Briefly, the main purpose is to check the robustness of the later analyses. If results from cosine-based retrieval or subspace overlap change a lot after standardisation, then part of the original result may have been driven by rogue dimensions rather than by the overall geometry of the representation space.

Task 2.3 — Can the language be decoded?

27. For each layer, train a logistic-regression classifier predicting the language of a sentence from its representation (fit on dev, evaluate on devtest). Use a StandardScaler followed by LogisticRegression(max_iter=2000) in a scikit-learn Pipeline.

For each layer, every sentence is represented by one vector, so one sentence corresponds to one row of the data matrix. Since there are 997 dev sentences for each of the six languages, the training set contains 997×6 sentence vectors, and each vector has 768 dimensions.

Each row is also associated with a language label, such as English, French or Chinese. We then give these sentence representations and their labels to a classifier. A StandardScaler is first applied to the representations, followed by a LogisticRegression classifier, which learns from the dev set how to distinguish the six languages from their sentence vectors. The trained classifier is then evaluated on the devtest representations.

Code block

```
#27
# Load cached sentence representations
representations_dev = {lang: np.load(os.path.join(CACHE_DIR, f"{lang}_dev.npy")) for lang in LANGS}
representations_devtest = {lang: np.load(os.path.join(CACHE_DIR, f"{lang}_devtest.npy")) for lang in LANGS}

language_classification_accuracies = []

# Train and evaluate one classifier for each layer
for layer in range(13):
```

```
    # Combine the six languages into one training set
    training_representations = np.concatenate([representations_dev[lang][layer] for lang in LANGS])
    training_languages = np.concatenate([[lang] * len(representations_dev[lang][layer]) for lang in LANGS])
    # Combine the six languages into one evaluation set
    evaluation_representations = np.concatenate([representations_devtest[lang][layer] for lang in LANGS])
    evaluation_languages = np.concatenate([[lang] * len(representations_devtest[lang][layer]) for lang in LANGS])
    # Standardise the vectors, then train logistic regression
    classifier = Pipeline([
        ("scaler", StandardScaler()),
        ("logistic_regression", LogisticRegression(max_iter=2000))
    ])
     # Fit on dev and predict on devtest sentences
    classifier.fit(training_representations, training_languages)
    predicted_languages = classifier.predict(evaluation_representations)

    accuracy = accuracy_score(evaluation_languages, predicted_languages)
    language_classification_accuracies.append(accuracy)
    print(f"Layer {layer}: {accuracy}")
```

```
Layer 0: 1.0
Layer 1: 0.9998353096179183
Layer 2: 1.0
Layer 3: 1.0
Layer 4: 1.0
Layer 5: 1.0
Layer 6: 1.0
Layer 7: 1.0
Layer 8: 0.9998353096179183
Layer 9: 1.0
Layer 10: 1.0
Layer 11: 0.9998353096179183
Layer 12: 0.9998353096179183
```

28. Repeat after *per-language centring*: subtract from every vector the mean of its language, estimated on dev.

Exo28 repeats the same language-classification experiment as Exo27, but adds per-language centring before training the classifier.

For each layer and each language, we first compute the mean vector of its 997 dev sentence representations. Then this language-specific mean vector is subtracted from every sentence vector of that language. In this way, the whole vector cloud of each language is shifted so that its centre is at zero. The same mean estimated from dev is also subtracted from the corresponding devtest vectors.

This centring step is useful because, exo27 shows that the language can be predicted almost perfectly from the raw sentence representations, but it does not tell us what information the classifier is using. One possibility tested in exo28 is that part of this language information comes from differences in the mean positions of the language representations.

Per-language centring removes this possible mean offset by shifting the vector cloud of each language so that its centre is at zero. We can then repeat the same classification experiment and check whether the language is still predictable from the remaining structure of the representations.

Code block

```
#28
centred_language_classification_accuracies = []

for layer in range(13):
  centred_training_representations = []
  centred_evaluation_representations = []
  training_languages = []
  evaluation_languages = []

  # Centre each language separately using the mean estimated on dev
  for lang in LANGS:
      language_mean = representations_dev[lang][layer].mean(axis=0)

      centred_training_representations.append(representations_dev[lang][layer] - language_mean)
      centred_evaluation_representations.append(representations_devtest[lang][layer] - language_mean)

      training_languages.extend([lang] * len(representations_dev[lang][layer]))
      evaluation_languages.extend([lang] * len(representations_devtest[lang][layer]))

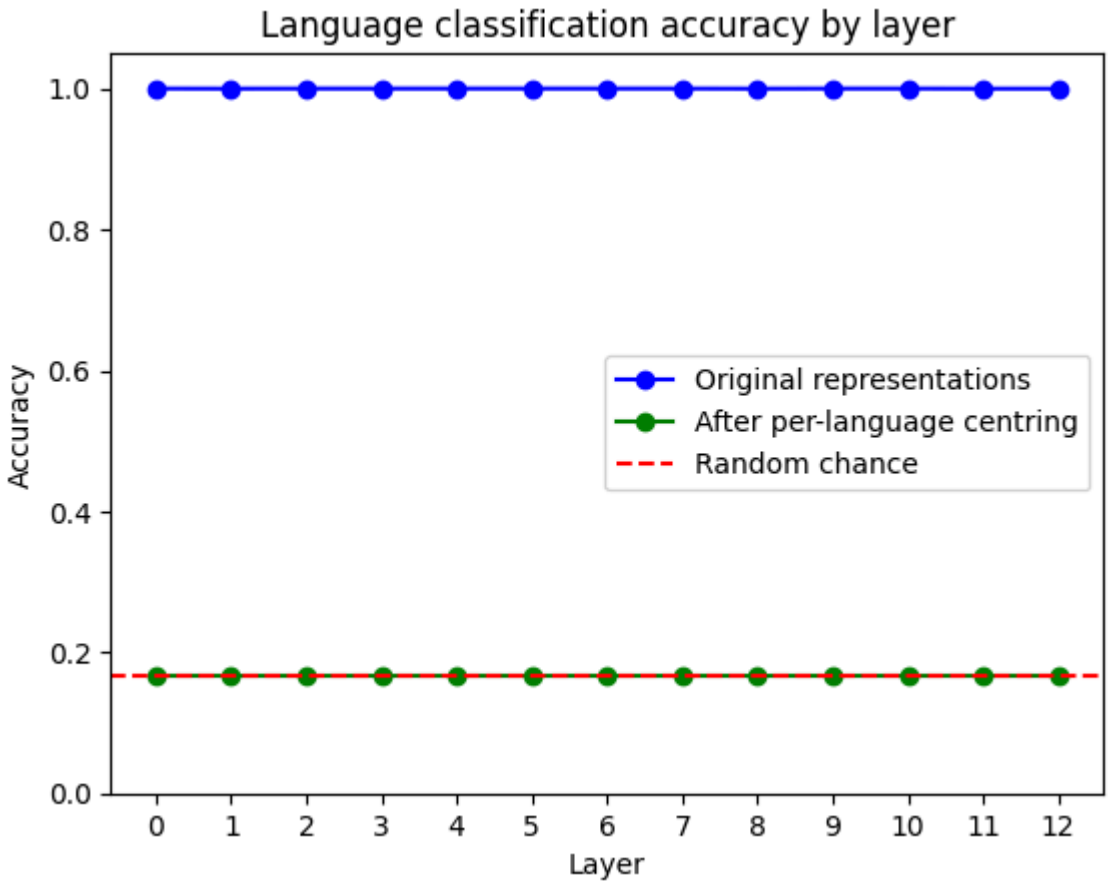
  # Combine the six languages
```

```
    centred_training_representations =
np.concatenate(centred_training_representations)
    centred_evaluation_representations =
np.concatenate(centred_evaluation_representations)

    # Same classifier as Exo27
    classifier = Pipeline([...
    print(f"Layer {layer}: {accuracy}")
```

> Layer 0: 0.16666666666666666
>
> Layer 1: 0.16666666666666666
>
> Layer 2: 0.16666666666666666
>
> Layer 3: 0.16666666666666666
>
> Layer 4: 0.16666666666666666
>
> Layer 5: 0.16666666666666666
>
> Layer 6: 0.16666666666666666
>
> Layer 7: 0.16666666666666666
>
> Layer 8: 0.16666666666666666
>
> Layer 9: 0.16666666666666666
>
> Layer 10: 0.16666666666666666
>
> Layer 11: 0.16666666666666666
>
> Layer 12: 0.16666666666666666

29. Plot accuracy against the layer for both conditions, with the chance level.



30. What is the chance level?

There are six language classes and they are balanced, so the chance level is 1/6 , about 16.7%.

31. Accuracy is probably close to 100% at every layer. Why does this result say nothing about the disjoint-subspace hypothesis? How many dimensions does a linear classifier need to separate *L* classes with distinct means?

In Exo 27, the language classification accuracy is close to 100% at every layer. This shows that the language of a sentence is easily recoverable from its representation with a linear classifier. However, this does not mean that different languages occupy disjoint subspaces.

The reason is that linear separability is a weaker condition than having disjoint subspaces. The representations of different languages could occupy largely the same directions in the representation space, while their vector clouds are centred at different positions. If these mean positions are sufficiently separated relative to the variation within each language, a linear classifier can distinguish the languages using these differences in position, even if their underlying subspaces strongly overlap.

For L classes with distinct mean vectors, we can choose one mean as a reference and describe the other class means using L-1 difference vectors. Therefore, the relative positions of the L class means lie in a subspace of at most L-1 dimensions. This does'not guarantee perfect classification, because the result also depends on how separated the class distributions are, but these L-1 directions can be sufficient when the means are well separated.

In our experiment, there are six languages, so the differences between their mean vectors span at most 6-1=5 dimensions. This is very small compared with the full 768 dimensional mBERT representation space. So an accuracy close to 100% can be explained by a small number of mean-difference directions and does not provide evidence for the disjoint-subspace hypothesis.

32. How much does accuracy drop after per-language centring? Explain why a linear classifier can hardly beat chance when the class means coincide, and what an accuracy remaining well above chance would reveal.

After per-language centring, the classification accuracy drops from almost 100% to about 16.7% at every layer. 16.7% is the chance level for six balanced language classes.

A linear classifier mainly separates classes by finding directions along which their representations have different positions. Before centring, different languages may have different mean vectors, so the classifier can use these mean offsets as strong cues for language identification. After per-language centring, however, the mean of every language is moved to the same position. So these systematic differences in mean position disappear, and a linear classifier can no longer rely on them. If the remaining differences between languages are mainly related to the spread or shape of their distributions rather than their mean positions, a linear decision boundary can hardly exploit them, so the accuracy is expected to fall close to chance.

In our results, the accuracy falls exactly to the chance level, which shows that most of the language information accessible to the linear classifier in exo 27 came from differences between the language means.

If the accuracy had remained well above chance after centring, it would mean that language information is not only encoded in the different mean positions of the languages. Even after all language means are aligned, the sentence representations would still show systematic differences along some directions that a linear classifier can use to distinguish the languages.

Task 2.4 — Is language identity just an offset?

For each language X, English and each layer, perform cross-lingual retrieval on devtest: for each English sentence, find the most similar sentence (cosine similarity) among the 1,012 sentences in X, and compute precision at 1 (the proportion of English sentences whose nearest neighbour is their translation). Compare three variants: raw, per-language centred (means estimated on dev), and centred after standardisation (this last variant only if you did Task 2.2).

For each layer, we use the English devtest sentence representations as queries, and compare each of them with 1,012 sentence representations of a target language using cosine similarity. For each English sentence, we select the target sentence with the highest cosine similarity and check whether it is the correct translation. Precision at 1 is then the proportion of English sentences for which the top-ranked target sentence is the true translation.

We perform this retrieval twice: first with the raw representations, and then with per-language centred representations. For the centred condition, the mean of each language is estimated on dev and subtracted from the corresponding devtest representations before retrieval.

Code block

```
def compute_precision_at_1(source_vectors: np.ndarray, target_vectors: np.ndarray) -> float:
    """Compute Precision at 1 for parallel sentence retrieval using cosine similarity."""

    # Normalise each sentence vector to unit length
    source_normalised = source_vectors / np.linalg.norm(source_vectors, axis=1, keepdims=True)
    target_normalised = target_vectors / np.linalg.norm(target_vectors, axis=1, keepdims=True)

    cosine_similarities = source_normalised @ target_normalised.T
    nearest_neighbour_indices = np.argmax(cosine_similarities, axis=1)
    # Parallel sentences have the same index in devtest
    correct_translation_indices = np.arange(len(source_vectors))

    # Proportion whose top-1 neighbour is the true translation
    return np.mean(nearest_neighbour_indices == correct_translation_indices)

english = "eng_Latn"
target_languages = [lang for lang in LANGS if lang != english]

raw_retrieval_precision_at_1 = {lang: [] for lang in target_languages}
centred_retrieval_precision_at_1 = {lang: [] for lang in target_languages}

for layer in range(13):
    # English representations at this layer
    english_dev = representations_dev[english][layer]
    english_devtest = representations_devtest[english][layer]
    # Estimate the English mean on dev
    english_mean = english_dev.mean(axis=0)
    # Centre English devtest representations using the dev mean
    english_devtest_centred = english_devtest - english_mean

    for lang in target_languages:
        # Target-language representations at this layer
        target_dev = representations_dev[lang][layer]
        target_devtest = representations_devtest[lang][layer]
        # Raw retrieval
        raw_precision = compute_precision_at_1(
            english_devtest,
```

```
            target_devtest
        )
        raw_retrieval_precision_at_1[lang].append(raw_precision)

        # Per-language centred retrieval :
        # Estimate the target-language mean on dev
        target_mean = target_dev.mean(axis=0)

        # Centre target devtest representations using the dev mean
        target_devtest_centred = target_devtest - target_mean

        centred_precision = compute_precision_at_1(
            english_devtest_centred,
            target_devtest_centred
        )
        centred_retrieval_precision_at_1[lang].append(centred_precision)
```

eng_Latn -> fra_Latn

Layer 0: raw = 0.434, centred = 0.882

Layer 1: raw = 0.519, centred = 0.939

Layer 2: raw = 0.770, centred = 0.958

Layer 3: raw = 0.892, centred = 0.977

Layer 4: raw = 0.955, centred = 0.990

Layer 5: raw = 0.981, centred = 0.992

Layer 6: raw = 0.992, centred = 0.995

Layer 7: raw = 0.995, centred = 0.994

Layer 8: raw = 0.993, centred = 0.996

Layer 9: raw = 0.992, centred = 0.996

Layer 10: raw = 0.982, centred = 0.991

Layer 11: raw = 0.970, centred = 0.985

Layer 12: raw = 0.978, centred = 0.975

eng_Latn -> deu_Latn

Layer 0: raw = 0.275, centred = 0.830

Layer 1: raw = 0.408, centred = 0.906

Layer 2: raw = 0.634, centred = 0.943

Layer 3: raw = 0.813, centred = 0.960

Layer 4: raw = 0.941, centred = 0.990

Layer 5: raw = 0.978, centred = 0.992

Layer 6: raw = 0.991, centred = 0.995

Layer 7: raw = 0.996, centred = 0.997

Layer 8: raw = 0.994, centred = 0.996

Layer 9: raw = 0.989, centred = 0.995

Layer 10: raw = 0.984, centred = 0.991

Layer 11: raw = 0.968, centred = 0.983

Layer 12: raw = 0.978, centred = 0.987

eng_Latn -> rus_Cyrl

Layer 0: raw = 0.068, centred = 0.615

Layer 1: raw = 0.257, centred = 0.737

Layer 2: raw = 0.538, centred = 0.805

Layer 3: raw = 0.688, centred = 0.880

Layer 4: raw = 0.838, centred = 0.944

Layer 5: raw = 0.931, centred = 0.973

Layer 6: raw = 0.962, centred = 0.981

Layer 7: raw = 0.980, centred = 0.986

Layer 8: raw = 0.989, centred = 0.993

Layer 9: raw = 0.973, centred = 0.985

Layer 10: raw = 0.974, centred = 0.976

Layer 11: raw = 0.952, centred = 0.946

Layer 12: raw = 0.935, centred = 0.944

eng_Latn -> cmn_Hans

Layer 0: raw = 0.120, centred = 0.758

Layer 1: raw = 0.224, centred = 0.836

Layer 2: raw = 0.588, centred = 0.869

Layer 3: raw = 0.725, centred = 0.929

Layer 4: raw = 0.868, centred = 0.963

Layer 5: raw = 0.935, centred = 0.977

Layer 6: raw = 0.962, centred = 0.988

Layer 7: raw = 0.977, centred = 0.989

Layer 8: raw = 0.981, centred = 0.987

Layer 9: raw = 0.972, centred = 0.986

Layer 10: raw = 0.969, centred = 0.977

Layer 11: raw = 0.963, centred = 0.968

Layer 12: raw = 0.950, centred = 0.959

eng_Latn -> yor_Latn

Layer 0: raw = 0.022, centred = 0.153

Layer 1: raw = 0.011, centred = 0.148

Layer 2: raw = 0.010, centred = 0.170

Layer 3: raw = 0.008, centred = 0.202

Layer 4: raw = 0.008, centred = 0.210

Layer 5: raw = 0.016, centred = 0.255

Layer 6: raw = 0.028, centred = 0.275

Layer 7: raw = 0.057, centred = 0.273

Layer 8: raw = 0.074, centred = 0.298

Layer 9: raw = 0.038, centred = 0.280

Layer 10: raw = 0.037, centred = 0.228

Layer 11: raw = 0.036, centred = 0.213

Layer 12: raw = 0.087, centred = 0.200

33. What is the precision at 1 of a random retrieval?

Since there are 1,012 candidate sentences and only one correct translation for each English sentence, the precision at 1 of a random retrieval is 1/1012, which is about about 0.10%.

34. Interpret the effect of centring in terms of the distinction between linear and affine sub spaces (see ‘From a vague hypothesis to testable predictions’).

Centring improves cross-lingual retrieval most clearly in the first few layers, roughly from layers 0 to 4.

At layer 0, for example, Precision at 1 (P@1) increases from 0.434 to 0.882 for French, from 0.275 to 0.830 for German, from 0.068 to 0.615 for Russian, and from 0.120 to 0.758 for Chinese. From about layers 5–10, the raw retrieval scores are already close to 1.0 for these languages, so centring brings only a small additional improvement.

One way to understand this is to write a representation approximately as $x = z + \mu_L$, where z contains the content-related part and μ_L is a language-specific mean offset. Different languages can therefore have similar underlying content directions while their whole vector clouds are shifted to different positions.

Centring removes this shift. The internal shape of each cloud is unchanged, but the vectors now have different directions relative to the origin. Since cosine similarity depends on these directions, removing the language offset can make translation pairs much easier to retrieve.

The large improvement after centring therefore suggests that a substantial part of the difference between languages is affine, coming from different mean positions rather than completely different linear content subspaces. However, the offset is not the whole story. Yoruba also improves after centring, but its P@1 remains much lower, for example from 0.022 to 0.153 at layer 0 and from 0.074 to 0.298 at layer 8, which shows its weaker alignment with English cannot be explained by the mean offset alone.

35. At which layers is retrieval best? Compare with the literature (Pires, Schlinger and Garrette 2019; Libovický, Rosa and Fraser 2020).

Retrieval works best in the middle-to-upper layers, especially around layers 7–9. If we average the results over the five target languages, both raw and centred retrieval reach their best score at layer 8. The average P@1 is about 0.806 for the raw representations and 0.854 after centring.

For French, German, Russian and Chinese, retrieval becomes much better from the lower layers to the middle layers, stays very high around layers 7–9, and then drops slightly in the last layers. Yoruba is different because its scores are much lower overall, but its best centred result is also at layer 8.

This is close to what was found in previous studies. Pires et al. (2019) also observed that cross-lingual nearest-neighbour retrieval becomes better in the middle layers and then decreases again near the final layers. Libovický, Rosa and Fraser (2020) found an even more similar result: in their sentence-retrieval experiment, the 8th layer gave the best performance for mBERT, and centring also improved retrieval.

So our results show the same general pattern: cross-lingual sentence alignment is strongest around layer 8, rather than in the embedding layer or in the final layers.

36. How does your low-resource language compare with the others? If you did Part 1, relate your answer to it.

Yoruba performs much worse than the other target languages in cross-lingual retrieval. Its raw P@1 stays very low across all layers, between about 0.008 and 0.087, while French, German, Russian and Chinese reach values close to 1.0 in the middle layers.

Centring helps Yoruba clearly, but the improvement is still much smaller than for the other languages. Its centred P@1 rises to roughly 0.15–0.30, with the best result around layer 8 at 0.298. This is still far below the scores of the other languages. So Yoruba shows much weaker cross-lingual alignment with English.

Task 2.5 — Measure the overlap between principal subspaces

38. Implement $c(X, U)$ and $\omega k(A \to B)$ (Eq. 2). Obtain principal subspaces with np.linalg.svd(Xc, full_matrices=False), and a random orthonormal basis from the QR decomposition of a Gaussian matrix of shape (d, k).

I implemented four functions for measuring the overlap between principal subspaces.

- compute_principal_subspace() uses SVD on centred sentence representations and keeps the top k directions with the largest variance.
- compute_captured_variance() computes c(X,U), which measures how much of the variance in X is captured by a given subspace U.
- create_random_subspace() creates a random k-dimensional orthonormal subspace using a Gaussian matrix followed by QR decomposition.
- compute_normalised_overlap() computes $\omega_k(A \to B)$ by comparing how much of language B's variance is captured by language A's principal subspace, relative to a random subspace and B's own principal subspace.

Code block

```
1  def compute_principal_subspace(centred_representations: np.ndarray,
   number_of_dimensions: int) -> np.ndarray:
2      #Find the top-k principal directions of centred sentence representations.
3      _, _, right_singular_vectors = np.linalg.svd(centred_representations,
   full_matrices=False)
4      return right_singular_vectors[:number_of_dimensions].T  # shape: (768, k)
5
6  def compute_captured_variance(centred_representations: np.ndarray,
   subspace_basis: np.ndarray) -> float:
7      # Compute c(X, U), the proportion of variance in X captured by subspace U.
```

```
    projected_representations = centred_representations @ subspace_basis
    return np.sum(projected_representations ** 2) /
np.sum(centred_representations ** 2)

def create_random_subspace(representation_dimension: int,
number_of_dimensions: int, seed: int = 42) -> np.ndarray:
    #Create a random k-dimensional orthonormal subspace for the random
baseline.
    random_generator = np.random.default_rng(seed)
    gaussian_matrix = random_generator.normal(size=(representation_dimension,
number_of_dimensions))
    random_subspace, _ = np.linalg.qr(gaussian_matrix)
    return random_subspace

def compute_normalised_overlap(
    target_representations: np.ndarray,
    source_subspace: np.ndarray,
    target_subspace: np.ndarray,
    random_subspace: np.ndarray
) -> float:
    """
    Compute omega_k(A -> B), measuring how well A's principal subspace
captures B's variance
    compared with a random subspace and B's own principal subspace.
    """
    source_captured_variance =
compute_captured_variance(target_representations, source_subspace)
    target_captured_variance =
compute_captured_variance(target_representations, target_subspace)
    random_captured_variance =
compute_captured_variance(target_representations, random_subspace)

    return (source_captured_variance - random_captured_variance) /
(target_captured_variance - random_captured_variance)
```

39. For $k \in \{10, 50, 100\}$ and every layer, compute ωk for all ordered pairs of distinct languages, on raw representations (and on standardised ones if you did Task 2.2).

For each layer, I first centred the dev and devtest representations of each language using the mean estimated on dev. I then applied SVD to the centred dev representations to obtain the principal directions of each language.

The largest subspace size used in this experiment is 100 dimensions. So I computed the first 100 principal directions only once for each language and each layer. When using 10 or 50 dimensions, I simply selected the first 10 or first 50 directions from these 100 instead of running SVD again.

I then computed the normalised overlap for the three subspace sizes, 10, 50 and 100 dimensions, and for every ordered pair of different languages. For each pair, the principal subspace of the source language was evaluated on the centred devtest representations of the target language. A random subspace was used as the lower reference, while the target language's own principal subspace was used as the upper reference.

There are six languages, so each language is compared with the other five languages. This gives 30 ordered language pairs for every layer and every subspace size. Since I did not do Task 2.2, I only used the raw, non-standardised representations.

Code block

```
#39
subspace_dimensions = [10, 50, 100]
maximum_dimensions = max(subspace_dimensions)
representation_dimension = representations_dev[LANGS[0]].shape[-1]

# One random subspace for each value of k
random_subspaces = {
    k: create_random_subspace(representation_dimension, k, seed=42 + k)
    for k in subspace_dimensions
}

overlap_results = []

for layer in range(13):
    centred_dev = {}
    centred_devtest = {}
    principal_subspaces = {}

    # Centre each language using its dev mean and find its top 100 directions
    for lang in LANGS:
        language_mean = representations_dev[lang][layer].mean(axis=0)

        centred_dev[lang] = representations_dev[lang][layer] - language_mean
        centred_devtest[lang] = representations_devtest[lang][layer] -
language_mean

        principal_subspaces[lang] = compute_principal_subspace(
            centred_dev[lang], maximum_dimensions
        )

    # Compare every ordered pair A -> B for k = 10, 50, 100
    for k in subspace_dimensions:
        random_subspace = random_subspaces[k]

        for source_language in LANGS:
```

```
            for target_language in LANGS:
                if source_language == target_language:
                    continue
                source_subspace = principal_subspaces[source_language][:, :k]
                target_subspace = principal_subspaces[target_language][:, :k]

                overlap = compute_normalised_overlap(
                    centred_devtest[target_language],
                    source_subspace,
                    target_subspace,
                    random_subspace
                )
                overlap_results.append({
                    "layer": layer,
                    "k": k,
                    "source_language": source_language,
                    "target_language": target_language,
                    "omega": overlap
                })
overlap_results = pd.DataFrame(overlap_results)
print(overlap_results.shape)
overlap_results.head()
```

40. Plot ωk averaged over pairs against the layer, separating pairs that share a script from pairs that do not. At the layer of your choice, show the full 6×6 matrix as a heat map.

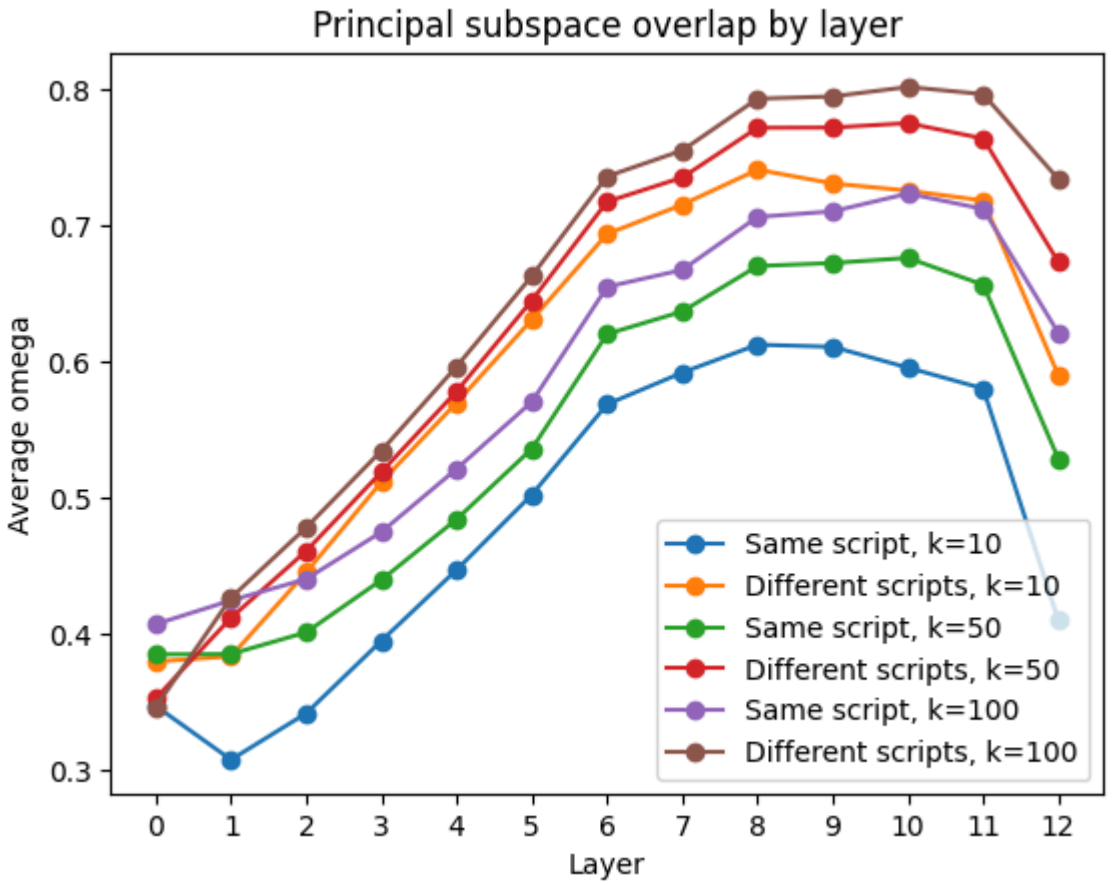
English, French, German and Yoruba use the Latin script, Russian uses the Cyrillic script, and Chinese uses the Han script. Based on this information, I divided the ordered language pairs into two groups. If the source and target languages use the same script, the pair is placed in the same-script group. Otherwise, it is placed in the different-script group.

For each layer and each subspace size, I then collected the overlap values of all language pairs belonging to the same group and computed their average. This gives, for every layer, one average overlap value for same-script pairs and one for different-script pairs. I plotted these averages across the 13 layers to compare how the principal-subspace overlap changes through the model.

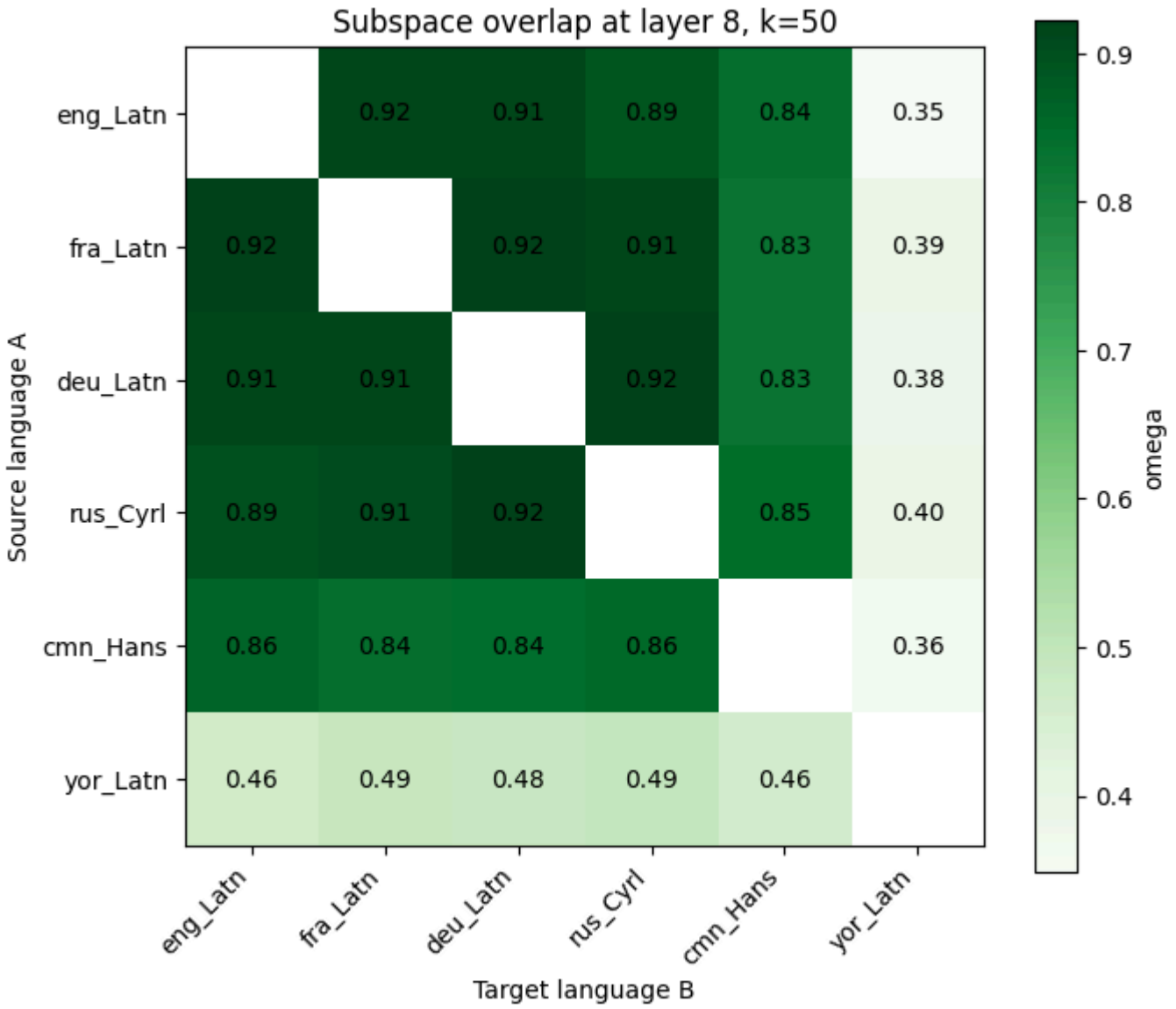
For the heat map, I selected layer 8 and a subspace size of 50 dimensions. Instead of averaging the language pairs, I kept the individual overlap value for each source-target pair and arranged them in a 6×6 matrix. Each cell shows how well the principal subspace of one language captures the variation of another language.

For the full code, see the Colab notebook : colab.research.google.com

	layer	k	script_group	omega
0	0	10	Different scripts	0.37972
1	0	10	Same script	0.347069
2	0	50	Different scripts	0.352744
3	0	50	Same script	0.384925
4	0	100	Different scripts	0.346306



target_language	eng_Latn	fra_Latn	deu_Latn	rus_Cyrl	cmn_Hans	yor_La
source_language						
eng_Latn	NaN	0.915982	0.913402	0.89003	0.839716	0.348
fra_Latn	0.92282	NaN	0.918785	0.911999	0.828778	0.389
deu_Latn	0.911498	0.910391	NaN	0.9188	0.830017	0.3785
rus_Cyrl	0.893117	0.90718	0.922084	NaN	0.848473	0.3952
cmn_Hans	0.857811	0.836654	0.842953	0.857028	NaN	0.3649
yor_Latn	0.464751	0.48687	0.483123	0.490906	0.458444	Na



41. Show that, for any X, the expected value of $c(X, U\text{rand})$ is k/d. (Hint: $\mathrm{E}[U\text{rand}\,U\text{rand}^\top] = \frac{k}{d} I_d$.) Check it numerically.

For the numerical check, I reused the functions defined in exo38, especially create_random_subspace() and compute_captured_variance().

I selected the English representations from layer 8 as an example. The dev mean was first used to centre the English devtest representations. Then, for each subspace size of 10, 50 and 100 dimensions, I generated 500 different random orthonormal subspaces.

For every random subspace, I used compute_captured_variance() to calculate how much of the variance of the centred English representations was captured. I then averaged the 500 values to approximate the expected captured variance.

Finally, I compared this numerical average with the theoretical value given by the subspace size divided by the full representation dimension, which is 768.

Code block

```python
language = "eng_Latn"
layer = 8
number_of_trials = 500

language_mean = representations_dev[language][layer].mean(axis=0)
centred_representations = representations_devtest[language][layer] - language_mean
representation_dimension = centred_representations.shape[1]

for k in [10, 50, 100]:
    captured_variances = []

    # Generate 500 random subspaces and compute c(X, U_rand)
    for trial in range(number_of_trials):
        random_subspace = create_random_subspace(
            representation_dimension,
            k,
            seed=trial
        )

        captured_variances.append(
            compute_captured_variance(
                centred_representations,
                random_subspace
            )
        )

    numerical_value = np.mean(captured_variances)
    theoretical_value = k / representation_dimension

    print(f"k={k}: numerical={numerical_value:.4f}, "
          f"theoretical={theoretical_value:.4f}")
```

> k=10: numerical=0.013050652852935333, theoretical=0.013020833333333334
>
> k=50: numerical=0.06511098058721822, theoretical=0.06510416666666667
>
> k=100: numerical=0.13027926176013357, theoretical=0.13020833333333334

The captured variance of a subspace is defined as $c(X, U) = \frac{|\bar{X}U|_F^2}{|\bar{X}|_F^2}$, where $\bar{X}$ is the centred representation matrix and U is an orthonormal basis of the subspace.

For a random k-dimensional subspace in a d-dimensional space, the question gives $\mathbb{E}[U_{\text{rand}}U_{\text{rand}}^T] = \frac{k}{d}I_d$. If we generate many random k-dimensional subspaces, no particular direction is preferred. Since the full representation space has d dimensions and the random

subspace contains k dimensions, it keeps, on average, a proportion k/d of the original variation. Therefore, the expected proportion of variance captured by a random subspace is $\mathbb{E}[c(X, U_{\text{rand}})] = \frac{k}{d}$.

I then checked this result numerically using the English representations from layer 8. For each subspace size, I generated 500 random orthonormal subspaces and computed the captured variance for each one. The average values were:

- 10 dimensions: 0.0131 numerically, compared with 0.0130 theoretically.
- 50 dimensions: 0.0651 numerically, compared with 0.0651 theoretically.
- 100 dimensions: 0.1303 numerically, compared with 0.1302 theoretically.

The numerical results are very close to the theoretical ones. It shows that in practice, a random k-dimensional subspace captures about k/d of the total variance on average. The implementation is consistent with the theoretical expectation.

42. Why is the ceiling computed with U_B fitted on dev and evaluated on devtest? What would happen if you fitted and evaluated it on the same sentences?

U_B is fitted on the dev set and evaluated on devtest because we want to know whether the principal subspace of language B also works well on new sentences.

If we used the same sentences to fit and evaluate U_B, the result would be optimistic. SVD directly finds the directions that explain as much variance as possible in the data used to fit it, so testing on the same data would give an artificially high ceiling.

Using dev for fitting and devtest for evaluation avoids this problem and gives a fairer estimate of how well the subspace generalises.

43. How does ωk vary with k? What happens when k approaches d? Why must k remain much smaller than the number of sentences used to estimate UA?

- From the plot, ω_k generally becomes larger when k increases. From around layer 2 onward, the overlap is usually highest for k=100, followed by k=50 and then k=10. It means that when we include more principal directions, the subspaces of different languages tend to overlap more. However, this pattern is not perfectly regular in the first layers.
- When k gets close to the full representation dimension d=768, the subspaces become closer to the whole representation space. As a result, the source subspace, the target subspace and even a random subspace would all capture almost all of the variance. At k=d, they all represent the full space, so the overlap measure is no longer useful.

- k should stay much smaller than the number of sentences used to estimate the principal subspace. If k is too large compared with the amount of data, we start including weaker principal directions that may only describe particular details of the dev sentences, rather than stable patterns that would also appear in new sentences.. Here we have 997 dev sentences per language, so using 10, 50 and 100 dimensions keeps the subspaces much smaller than the amount of available data.

44. Is ω symmetric? Interpret an asymmetry between English and your low-resource language.

ω is not symmetric. In the heat map from exo 40, at layer 8 with 50 dimensions, the overlap from English to Yoruba is about 0.348, while the overlap from Yoruba to English is about 0.465.

This is because the two directions are not measuring the same thing. For English -> Yoruba, we use the English principal subspace to see how much of the Yoruba variation it can capture. For Yoruba -> English, we do the opposite and use the Yoruba principal subspace to explain the English representations.

The difference between the two values suggests that the main directions found for English and Yoruba do not match in exactly the same way in both directions. In this example, the Yoruba principal subspace captures English variation better than the English principal subspace captures Yoruba variation.

And this shows that the overlap depends on which language is used as the source and which one is used as the target.

45. At which layers is the overlap lowest? Is a low overlap at layer 0 between languages with different scripts evidence for H_{disjoint} ?

- From the plot in exo40, the overlap is generally lowest in the first layers, especially around layers 0 and 1. Most curves start with relatively low values and then increase clearly from the following layers. One small exception is the same-script condition with 10 dimensions, where the lowest value appears at layer 1 rather than layer 0.
- A low overlap at layer 0 between languages with different scripts is NOT enough to support H_{disjoint} . Layer 0 mainly contains the input embeddings before contextual processing. Languages using different scripts also use very different WordPieces and token embeddings, so their representations can naturally have different main directions at this stage. Therefore, the low overlap at layer 0 may mainly reflect differences in script and lexical representation, rather than showing that the languages use completely separate content subspaces.

46. (Only if you did Task 2.2.) Does standardisation change your conclusions? What does this tell you about rogue dimensions?

47. Suppose $\omega_k \approx 1$ in all middle layers. Does this establish that mBERT represents all languages with shared features? Use ‘The superposition hypothesis’.

No. Even if ω_k is close to 1 in the middle layers, this only shows that the principal subspaces of the languages are very similar. It doesn't show that the same directions represent the same features in every language.

According to the superposition hypothesis, different language-specific features can still use the same dimensions. For example, a direction could be related to one feature in English and to a different feature in Russian, while both languages still have similar principal subspaces.

So a high overlap is compatible with shared features, but it is also compatible with different language-specific features being superposed in the same subspace. We need another experiment to check whether the same direction has the same meaning across languages.

Task 2.6 — Same direction, same meaning?

We now test whether a direction that encodes some property in one language encodes the same property in another. We use the domain field of FLORES+ (three classes), which is identical for a sentence and its translations.

48. At two or three layers of your choice, train a logistic-regression probe predicting the do main on the dev representations of a language *A*. Evaluate it on the devtest representations of *A* (within-language accuracy) and of every other language *B* (transfer accuracy). Report the majority-class baseline.

Code block

```
source_language = "eng_Latn"
selected_layers = [0, 8, 12]

training_labels = dev[source_language]["domain"].to_numpy()

# Majority-class baseline
majority_class = pd.Series(training_labels).value_counts().idxmax()
majority_baseline = np.mean(
    devtest[source_language]["domain"].to_numpy() == majority_class
)

results = []

for layer in selected_layers:
    # Train only on English dev
    X_train = representations_dev[source_language][layer]

    classifier = LogisticRegression(max_iter=2000)
```

```
    classifier.fit(X_train, training_labels)

    # Evaluate the same classifier on every language
    for target_language in LANGS:
        X_test = representations_devtest[target_language][layer]
        y_test = devtest[target_language]["domain"].to_numpy()

        predictions = classifier.predict(X_test)
        accuracy = accuracy_score(y_test, predictions)

        results.append({
            "layer": layer,
            "source": source_language,
            "target": target_language,
            "accuracy": accuracy
        })

results = pd.DataFrame(results)
results
```

> Majority class: wikinews
>
> Majority-class baseline: 0.337

	layer	source	target	accuracy
0	0	eng_Latn	eng_Latn	0.712451
1	0	eng_Latn	fra_Latn	0.504941
2	0	eng_Latn	deu_Latn	0.589921
3	0	eng_Latn	rus_Cyrl	0.48913
4	0	eng_Latn	cmn_Hans	0.547431
5	0	eng_Latn	yor_Latn	0.397233
6	8	eng_Latn	eng_Latn	0.779644
7	8	eng_Latn	fra_Latn	0.75
8	8	eng_Latn	deu_Latn	0.713439
9	8	eng_Latn	rus_Cyrl	0.682806
10	8	eng_Latn	cmn_Hans	0.70751
11	8	eng_Latn	yor_Latn	0.405138
12	12	eng_Latn	eng_Latn	0.76581
13	12	eng_Latn	fra_Latn	0.706522
14	12	eng_Latn	deu_Latn	0.670949
15	12	eng_Latn	rus_Cyrl	0.631423
16	12	eng_Latn	cmn_Hans	0.718379
17	12	eng_Latn	yor_Latn	0.438735

I chose layers 0, 8 and 12. For each layer, I trained a logistic-regression probe on the English dev representations to predict the three domain labels. Then I evaluated the same probe on the English devtest set to obtain the within-language accuracy, and on the devtest representations of the other five languages to measure cross-lingual transfer. The majority class was wikinews, giving a baseline accuracy of 0.337.

For English, the within-language accuracy was 0.712 at layer 0, 0.780 at layer 8 and 0.766 at layer 12. Transfer was also clearly above the majority baseline for French, German, Russian and Chinese. At layer 8, the accuracy is 0.750 for French, 0.713 for German, 0.683 for Russian and 0.708 for Chinese. Yoruba is much lower, with accuracies between 0.397 and 0.439 across the three layers.

Overall, layer 8 gave the best transfer for most languages. The results show that the domain information learned from English can also be used in several other languages, although the transfer is much weaker for Yoruba.

49. Repeat with per-language centred representations.

Code block

```
source_language = "eng_Latn"
```

```
selected_layers = [0, 8, 12]

y_train = dev[source_language]["domain"].to_numpy()
y_test = devtest[source_language]["domain"].to_numpy()

centred_results = []

for layer in selected_layers:
    # Centre English training representations using English dev mean
    source_mean = representations_dev[source_language][layer].mean(axis=0)
    X_train = representations_dev[source_language][layer] - source_mean

    classifier = LogisticRegression(max_iter=2000)
    classifier.fit(X_train, y_train)

    for target_language in LANGS:
        # Centre each target language using its own dev mean
        target_mean = representations_dev[target_language][layer].mean(axis=0)
        X_test = representations_devtest[target_language][layer] - target_mean

        accuracy = classifier.score(X_test, y_test)

        centred_results.append([
            layer,
            source_language,
            target_language,
            accuracy
        ])

centred_results = pd.DataFrame(
    centred_results,
    columns=["layer", "source", "target", "accuracy"]
)

centred_results
```

	layer	source	target	accuracy
0	0	eng_Latn	eng_Latn	0.712451
1	0	eng_Latn	fra_Latn	0.593874
2	0	eng_Latn	deu_Latn	0.586957
3	0	eng_Latn	rus_Cyrl	0.5583
4	0	eng_Latn	cmn_Hans	0.581028
5	0	eng_Latn	yor_Latn	0.394269
6	8	eng_Latn	eng_Latn	0.780632
7	8	eng_Latn	fra_Latn	0.748024
8	8	eng_Latn	deu_Latn	0.733202
9	8	eng_Latn	rus_Cyrl	0.725296
10	8	eng_Latn	cmn_Hans	0.725296
11	8	eng_Latn	yor_Latn	0.463439
12	12	eng_Latn	eng_Latn	0.764822
13	12	eng_Latn	fra_Latn	0.714427
14	12	eng_Latn	deu_Latn	0.683794
15	12	eng_Latn	rus_Cyrl	0.678854
16	12	eng_Latn	cmn_Hans	0.719368
17	12	eng_Latn	yor_Latn	0.425889

I repeated the experiment from exo48 after centring the representations of each language separately. For each language and layer, I computed the mean on the dev set, subtracted this mean from both dev and devtest representations, and then trained and evaluated the same logistic-regression probe as before.

50. Why are labels defined on the English source articles valid for every language?

The domain labels are valid across languages because all translations correspond to the same English source sentence and share the same domain.

51. Why are pairs of languages with different scripts the most informative here? (Think of what a probe could exploit when two languages share tokens, e.g. named entities or numbers.)

Language pairs with different scripts are more informative because they share fewer obvious lexical cues. For example, languages using the same script may share some words, named entities or numbers, and the probe could use these surface patterns to predict the domain. If the probe still transfers well to a language written in a different script, it is less likely that the result comes only from shared tokens. This gives stronger evidence that similar domain information is encoded in the representations across languages.

52. Why does centring matter for transfer?

Centring matters for transfer because the representations of different languages can have different mean positions in the space. A probe trained on English may partly depend on the English-specific offset, which is not present in the same way in another language. By subtracting each language' s own mean, we remove this global language-specific shift and compare the relative variation of the sentence representations instead. This makes it easier to test whether the domain-related directions learned in English are also useful in the other languages

53. Place your language pairs in the table below and say which scenario of 'The superposition hypothesis' your results support.

	transfer ≈ within-language	transfer ≈ baseline
high overlap ω	shared features	superposed specific features
low overlap ω	unlikely	disjoint subspaces

	Transfer ≈ within-language	**Transfer ≈ baseline**
High overlap omega	English–French, English–German, English–Russian, English–Chinese	—
Low overlap omega	—	English–Yoruba

Using layer 8 and $k = 50$, I compared the subspace overlap from exo40 with the centred transfer results from exo49. English–French, English–German, English–Russian and English–Chinese all have high overlap values, around 0.84 to 0.92. Their transfer accuracies are also very close to the English within-language accuracy of 0.781, so I place these pairs in the shared features case.

English–Yoruba is different. Its overlap is much lower, around 0.35, and its transfer accuracy is 0.463, which is closer to the majority baseline of 0.337 than to the English within-language accuracy. So I place it closer to the disjoint subspaces case, although the transfer is still above baseline, so the result is not completely clear.

Overall, most language pairs support the shared-features scenario. I don't observe a clear case of high overlap together with transfer close to baseline, so these results give little evidence for strongly superposed language-specific domain features.